

Transformación y escalamiento del dataset diario

Issue #52 — MA2003B, equipo 7

Tabla de contenidos

0. Carga y alcance	1
1. Las escalas son heterogéneas	3
2. Criterio $ \text{sesgo} < 0.5$	4
3. Box-Cox con desplazamiento	5
4. Estandarización z	7
5. La hipótesis del viento	8
El caso más extremo no es el viento	8
6. Distribuciones resultantes	9
7. Salida	12
8. Criterios de aceptación	12
9. Qué queda para el diccionario	12
Uso de IA en este notebook	12

Issue #52. Rehace sobre la tabla día-estación la transformación que hoy solo existe a resolución horaria: criterio $|\text{sesgo}| < 0.5 \rightarrow$ Box-Cox con desplazamiento $\rightarrow$ estandarización z. **Cambia el input, no el método:** el criterio y su justificación son los de `notebooks/transformaciones.qmd` (#35).

Opera sobre `sima_diario_modelado.parquet` (#51) y produce `sima_transformado_diario.parquet`, la entrada de #53 (correlación), #54 (PCA) y #55 (clasificación). Salda además una deuda: el `parquet` horario versionado salió del pipeline previo a la PR #46 y **no contiene** `N0`, `S02`, `RH` ni `SR`.

0. Carga y alcance

```
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
```

```
# No fijar matplotlib.use("Agg") aquí: es un backend no interactivo, así que
# plt.show() no emite nada, Quarto no captura ninguna figura y el PDF sale con
# los captions vacíos. Ya corregido en 04_eda_cualitativas.qmd (PR #50).

RAIZ = Path.cwd().parent
PROC = RAIZ / "data" / "processed"
FIGS = RAIZ / "reports" / "figuras"
FIGS.mkdir(parents=True, exist_ok=True)

# El cwd de Quarto es notebooks/, así que la raíz del repo no está en sys.path.
sys.path.insert(0, str(RAIZ))

D = pd.read_parquet(PROC / "sima_diario_modelado.parquet")
print(f"diario de modelado: {D.shape[0]:,} filas × {D.shape[1]} columnas")
```

diario de modelado: 26,691 filas × 34 columnas

De las 34 columnas se transforman **16**: exactamente las salidas de RESUMENES en `src/agregacion_diaria.py`, es decir los predictores continuos medidos. La lista se importa del módulo en vez de reescribirse, para que agregar un resumen allá no deje esta notebook desactualizada en silencio.

```
from src.agregacion_diaria import RESUMENES

CONTINUAS = [r.salida for r in RESUMENES]
print(len(CONTINUAS), "continuas:", CONTINUAS)
```

16 continuas: ['TOUT_max', 'SR_acum', 'NO_06_09', 'NO2_06_09', 'NOX_06_09', 'CO_06_09', 'WSR_me

Lo que queda fuera, y por qué:

Grupo	Columnas	Motivo
Binaria	llovio	Binaria; la issue excluye binarias y categóricas
Agrupación y calendario	zona, msnm, tipo_dia, festivo, temporada, mes, anio, fin_de_semana	Categóricas o de calendario
Objetivo	MDA8, ventanas_validas, O3_max_1h y las 4 indicadores de excedencia	Definen la clase
Clave	estacion, fecha	—

Dos casos de borde que el checklist de la issue no cubre y que se resuelven aquí:

- **msnm no se transforma.** Es continua, pero es un atributo de estación —15 valores repetidos a lo largo del dataset—, no una medición del día. Estandarizarla trataría como variación muestral lo que es una constante por grupo.
- **MDA8 y O3_max_1h no se estandarizan.** Son continuas pero definen la clase, no son predictores: `objetivo_tecnicas.tex` lo dice explícitamente. Se conservan en escala física (ppb) para que los umbrales de la NOM-020 sigan siendo legibles.

```
FUERA = [c for c in D.columns if c not in CONTINUAS]
assert len(CONTINUAS) + len(FUERA) == D.shape[1]
print(f"se transforman {len(CONTINUAS)}, se conservan intactas {len(FUERA)}")
print(FUERA)
```

se transforman 16, se conservan intactas 18

`['estacion', 'fecha', 'zona', 'msnm', 'anio', 'mes', 'temporada', 'tipo_dia', 'festivo', 'fin_c`

1. Las escalas son heterogéneas

PRS_media vive en torno a 700 hPa y SR_acum no pasa de 24; WSR_media llega a 148 y viento_u_media baja a −115. Sin escalamiento, el PCA de #54 y el discriminante de #55 quedarían dominados por las unidades, no por la señal.

```
plt.rcParams.update({"font.size": 9, "font.family": "serif",
                    "axes.spines.top": False, "axes.spines.right": False,
                    "figure.dpi": 150})
AZUL, ROJO, GRIS, CLARO = "#3b6ea5", "#b5443a", "#8c8c8c", "#d9d9d9"

fig, ax = plt.subplots(figsize=(6.3, 4.2))
datos = [D[c].dropna().values for c in CONTINUAS]
bp = ax.boxplot(datos, orientation="horizontal", tick_labels=CONTINUAS, widths=.6,
                flierprops=dict(marker=".", markersize=1.5, alpha=.25,
                                markerfacecolor=GRIS, markeredgecolor="none"),
                medianprops=dict(color=ROJO), patch_artist=True)
for caja in bp["boxes"]:
    caja.set(facecolor=CLARO, edgecolor=AZUL, linewidth=.8)
ax.set_xscale("symlog")
ax.set_xlabel("Escala original (symlog)", fontsize=8)
ax.tick_params(labelsize=7)
ax.invert_yaxis()
fig.tight_layout()
fig.savefig(FIGS / "transf_diaria_boxplot.pdf"); plt.show()
```

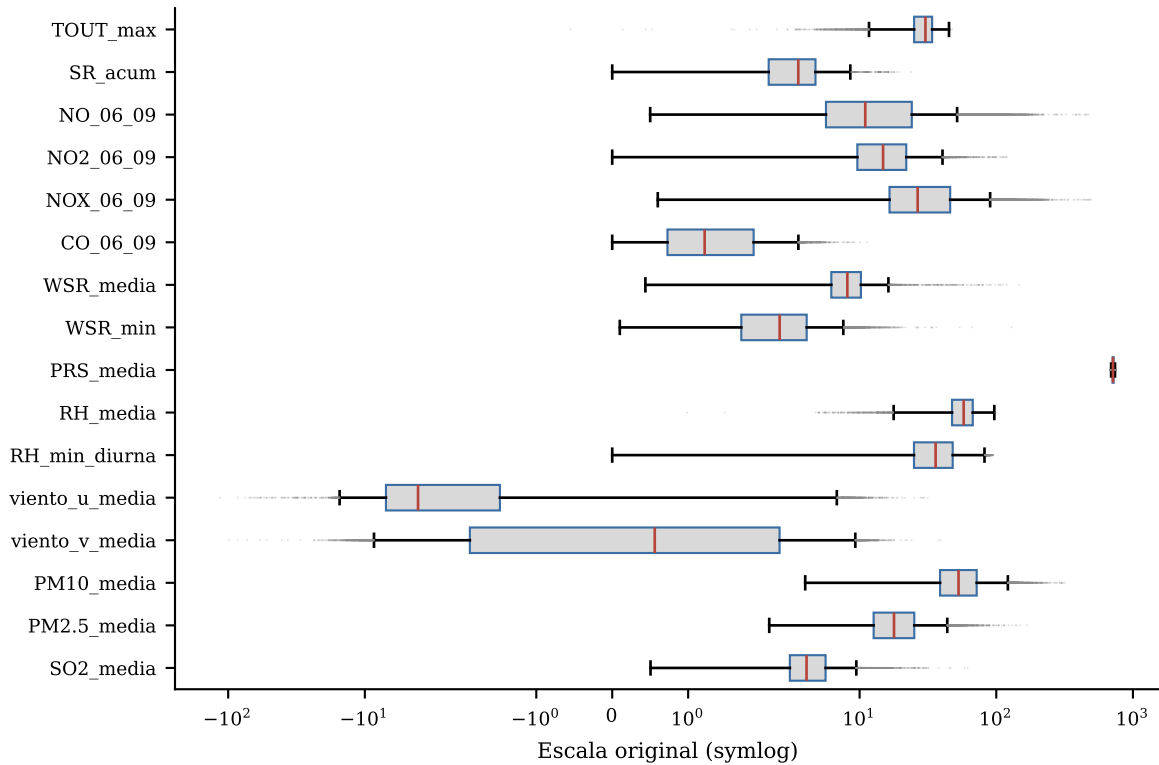


Figura 1: Escala original de las 16 continuas (eje symlog: hay valores negativos).

2. Criterio $|\text{sesgo}| < 0.5$

Se calcula el sesgo muestral de cada continua sobre sus valores no faltantes. Las que lo incumplen son las candidatas a Box-Cox; las que ya lo cumplen pasan directo a la estandarización, sin transformar.

```
UMBRAL = 0.5

base = pd.DataFrame({
    "n": [D[c].notna().sum() for c in CONTINUAS],
    "min": [D[c].min() for c in CONTINUAS],
    "max": [D[c].max() for c in CONTINUAS],
    "no_positivos": [int((D[c] <= 0).sum()) for c in CONTINUAS],
    "sesgo": [stats.skew(D[c].dropna()) for c in CONTINUAS],
    "curtosis": [stats.kurtosis(D[c].dropna()) for c in CONTINUAS],
}, index=CONTINUAS)
base["candidata"] = base.sesgo.abs() >= UMBRAL

CANDIDATAS = list(base.index[base.candidata])
print(base.round(3).to_string())
print(f"\ncandidatas: {len(CANDIDATAS)} de {len(CONTINUAS)}")
print("ya cumplen:", list(base.index[~base.candidata]))
```

	n	min	max	no_positivos	sesgo	curtosis	candidata
TOUT_max	25130	-0.550	47.580	1	-0.858	0.598	True
SR_acum	25568	0.000	23.790	2231	0.527	3.040	True
NO_06_09	24934	0.500	473.800	0	4.126	32.495	True
NO2_06_09	24915	0.000	119.425	2	1.666	6.893	True
NOX_06_09	24901	0.600	493.700	0	3.037	17.676	True
CO_06_09	24754	0.000	11.392	3	1.419	4.817	True
WSR_media	25683	0.437	147.632	0	8.293	149.737	True
WSR_min	25683	0.100	129.300	0	8.739	347.888	True
PRS_media	25805	681.276	747.945	0	0.116	-0.158	False
RH_media	24042	0.987	99.389	0	-0.367	0.190	False
RH_min_diurna	24116	0.000	95.000	3	0.446	0.153	False
viento_u_media	25244	-115.158	31.490	21568	-1.741	27.193	True
viento_v_media	25244	-98.956	39.134	11002	-1.516	27.682	True
PM10_media	25751	4.000	317.871	0	1.576	4.984	True
PM2.5_media	20726	2.182	167.979	0	2.038	9.863	True
SO2_media	24758	0.505	61.775	0	3.635	36.696	True

candidatas: 13 de 16

ya cumplen: ['PRS_media', 'RH_media', 'RH_min_diurna']

Trece de dieciséis incumplen. Las tres que no —PRS_media, RH_media, RH_min_diurna— son justamente las que la agregación diaria dejó bien portadas: presión y humedad varían poco dentro del día.

SR_acum merece una nota: tiene 2,231 ceros (días sin insolación registrada) y sale con sesgo 0.53, apenas por encima del umbral; con la primera versión de 2025 (cortada a junio) quedaba en 0.30 y no se transformaba. **No repite el caso de RAINF**, que a resolución horaria llegaba a 145 y hubo que dicotomizar. El acumulado diario, que es una suma sobre 24 h, diluye la inflación de ceros lo suficiente como para que la variable entre al modelado en escala continua, y Box-Cox la deja en -0.004 .

3. Box-Cox con desplazamiento

Box-Cox exige argumento estrictamente positivo. Para las candidatas que registran valores no positivos se aplica el mismo desplazamiento que #35: $x - \min(x) + 1$. El se estima por máxima verosimilitud con `scipy.stats.boxcox`.

```
def transformar(s: pd.Series, aplicar_bc: bool) -> tuple[pd.Series, float, float]:
    """Devuelve la serie transformada y estandarizada, más (desplazamiento, )."""
    m = s.notna()
    x = s[m].to_numpy(dtype=float)

    desplazamiento, lam = 0.0, np.nan
    if aplicar_bc:
        if x.min() <= 0:
            desplazamiento = -x.min() + 1.0
```

```

        x = x + desplazamiento
    x, lam = stats.boxcox(x)

    z = pd.Series(np.nan, index=s.index, name=s.name, dtype=float)
    z[m] = (x - x.mean()) / x.std(ddof=0)          # estandarización z (§4)
    return z, desplazamiento, lam

```

```

columnas, filas = {}, []
for c in CONTINUAS:
    z, desp, lam = transformar(D[c], c in CANDIDATAS)
    columnas[c] = z
    # El sesgo posterior se mide sobre z, pero z es una transformación afín: no
    # altera sesgo ni curtosis, así que la cifra es la de la escala Box-Cox.
    zv = z.dropna()
    filas.append({
        "variable": c, "n": int(zv.size), "box_cox": c in CANDIDATAS,
        "desplazamiento": desp, "lambda": lam,
        "sesgo_antes": base.loc[c, "sesgo"], "sesgo_despues": stats.skew(zv),
        "curtosis_antes": base.loc[c, "curtosis"],
        "curtosis_despues": stats.kurtosis(zv),
    })

params = pd.DataFrame(filas).set_index("variable")
params["cumple"] = params.sesgo_despues.abs() < UMBRAL
print(params.round(3).to_string())

```

variable	n	box_cox	desplazamiento	lambda	sesgo_antes	sesgo_despues	curtosis_antes
TOUT_max	25130	True	1.550	2.118	-0.858	-	
0.159	0.598		-0.492	True			
SR_acum	25568	True	1.000	0.755	0.527	-	
0.004	3.040		0.988	True			
NO_06_09	24934	True	0.000	-0.103	4.126	0.006	32.0
0.092	True						
NO2_06_09	24915	True	1.000	0.279	1.666	0.021	6.0
NOX_06_09	24901	True	0.000	0.008	3.037	0.001	17.0
CO_06_09	24754	True	1.000	-0.320	1.419	0.018	4.0
0.528	True						
WSR_media	25683	True	0.000	0.134	8.293	0.067	149.0
WSR_min	25683	True	0.000	0.285	8.739	0.044	347.0
PRS_media	25805	False	0.000	NaN	0.116	0.116	-
0.158	-0.158	True					
RH_media	24042	False	0.000	NaN	-0.367	-	
0.367	0.190		0.190	True			
RH_min_diurna	24116	False	0.000	NaN	0.446	0.446	0.0
viento_u_media	25244	True	116.158	2.955	-1.741	0.285	27.0
viento_v_media	25244	True	99.956	2.956	-1.516	0.195	27.0

PM10_media	25751	True	0.000	0.165	1.576	0.013
PM2.5_media	20726	True	0.000	0.021	2.038	0.000
SO2_media	24758	True	0.000	-0.202	3.635	-
0.013	36.696		0.465	True		

Las trece candidatas bajan de $|\text{sesgo}| < 0.5$, y las tres no transformadas ya estaban dentro. **Ninguna variable queda fuera del criterio**, a diferencia del horario, donde viento_u, viento_v y SR se quedaron en 0.75, 0.62 y 1.02.

4. Estandarización z

La estandarización va dentro de `transformar()`: se resta la media y se divide entre la desviación estándar poblacional (`ddof=0`) de los valores no faltantes de cada columna. Es requisito del PCA (#54) y del discriminante (#55).

Los faltantes se conservan como faltantes. No se imputan aquí: #51 ya decidió que el modelado descarta listwise, y rellenar antes de estandarizar contaminaría la media y la desviación con valores inventados.

```
T = D.assign(**columnas)
resumen_z = pd.DataFrame({"media": T[CONTINUAS].mean(),
                          "sd": T[CONTINUAS].std(ddof=0),
                          "faltantes": T[CONTINUAS].isna().sum()})
print(resumen_z.round(4).to_string())
```

	media	sd	faltantes
TOUT_max	-0.0	1.0	1561
SR_acum	0.0	1.0	1123
NO_06_09	0.0	1.0	1757
NO2_06_09	0.0	1.0	1776
NOX_06_09	-0.0	1.0	1790
CO_06_09	0.0	1.0	1937
WSR_media	-0.0	1.0	1008
WSR_min	0.0	1.0	1008
PRS_media	-0.0	1.0	886
RH_media	0.0	1.0	2649
RH_min_diurna	-0.0	1.0	2575
viento_u_media	0.0	1.0	1447
viento_v_media	-0.0	1.0	1447
PM10_media	-0.0	1.0	940
PM2.5_media	0.0	1.0	5965
SO2_media	-0.0	1.0	1933

5. La hipótesis del viento

La issue predice que, al promediar a día, las colas de ráfaga de `viento_u` y `viento_v` se aplanarían solas y la curtosis bajaría sin hacer nada. **Se cumple a medias.**

```
CURT_HORARIA = {"viento_u_media": 40.462, "viento_v_media": 40.078}
comp = pd.DataFrame({
    "curtosis_horaria": CURT_HORARIA,
    "curtosis_diaria": base.loc[list(CURT_HORARIA), "curtosis"],
    "curtosis_tras_bc": params.loc[list(CURT_HORARIA), "curtosis_despues"],
    "sesgo_diario": base.loc[list(CURT_HORARIA), "sesgo"],
    "sesgo_tras_bc": params.loc[list(CURT_HORARIA), "sesgo_despues"],
    "lambda": params.loc[list(CURT_HORARIA), "lambda"],
})
print(comp.round(3).to_string())
```

	curtosis_horaria	curtosis_diaria	curtosis_tras_bc	sesgo_diario	sesgo_tras_bc
viento_u_media	40.462	27.193	5.226	-	
1.741	0.285	2.955			
viento_v_media	40.078	27.682	4.858	-	
1.516	0.195	2.956			

Agregar a día baja la curtosis de exceso de 40.5 a 27.2 y de 40.1 a 27.7: una caída de apenas un tercio, que deja las dos variables tan leptocúrticas que ninguna técnica las trataría como normales. **La agregación por sí sola no resuelve el problema.**

Lo que sí cambia es que Box-Cox ahora funciona. A resolución horaria el sesgo era positivo y el estimado (1.9 y 1.5) amplificaba las colas; a resolución diaria el sesgo es **negativo** (−1.74 y −1.52, porque la media vectorial concentra masa en el lado negativo de u) y un $\lambda = 3$ es exactamente la corrección que pide una cola izquierda. El resultado: sesgo dentro del criterio y curtosis de ~28 a ~5.

Caveat para #55. Con ~5 de curtosis de exceso, el viento sigue siendo el par más pesado de la tabla junto con `WSR_media`, y el discriminante lineal sí es sensible a colas pesadas. Hay además una fragilidad de método: el desplazamiento es $-\min(x) + 1$, y ese mínimo lo fija **una sola fila** (SO2, 2021-08-06), así que depende de un dato extremo. Quedan marcadas como caso a vigilar, no como problema resuelto.

El caso más extremo no es el viento

```
print(base.curtosis.sort_values(ascending=False).head(4).round(1).to_string())
extremos = D.loc[D.WSR_min > 20, ["estacion", "fecha", "WSR_media", "WSR_min"]]
print(f"\ndías-estación con WSR_min > 20: {len(extremos)}")
print(extremos.estacion.value_counts().to_dict())
```



```
WSR_min      347.9
WSR_media    149.7
SO2_media     36.7
NO_06_09     32.5
```

```
días-estación con WSR_min > 20: 15
{'SO2': 11, 'NO3': 3, 'NE2': 1}
```

WSR_min tiene curtosis 348 y WSR_media 150, muy por encima del viento vectorial. El origen son 15 días-estación —11 de ellos SO2 en 2021— con velocidades de viento que **#13 conservó a propósito**: caen fuera del rango de operación anual del sensor pero dentro del rango de fabricante (0–180 km/h), y la política de limpieza fue marcarlos, no borrarlos.

No se tocan aquí: cambiar esa decisión es alcance de #13, no de #52. Box-Cox los comprime bien (curtosis 348 → 0.94 y 150 → 3.94) y quedan documentados para que #54 y #55 decidan con el número delante.

6. Distribuciones resultantes

```
fig, ax = plt.subplots(figsize=(6.3, 3.6))
rejilla = np.linspace(-6, 6, 400)
paleta = plt.cm.viridis(np.linspace(0, .92, len(CONTINUAS)))
for col, color in zip(CONTINUAS, paleta):
    v = T[col].dropna().to_numpy()
    ax.plot(rejilla, stats.gaussian_kde(v)(rejilla), color=color, lw=1)
ax.plot(rejilla, stats.norm.pdf(rejilla), color="black", lw=1.4, ls="--",
        label="N(0, 1)")
ax.set_xlabel("z-score", fontsize=8); ax.set_ylabel("densidad", fontsize=8)
ax.legend(fontsize=7, frameon=False)
ax.tick_params(labelsize=7)
fig.tight_layout()
fig.savefig(FIGS / "transf_diaria_densidad.pdf"); plt.show()
```

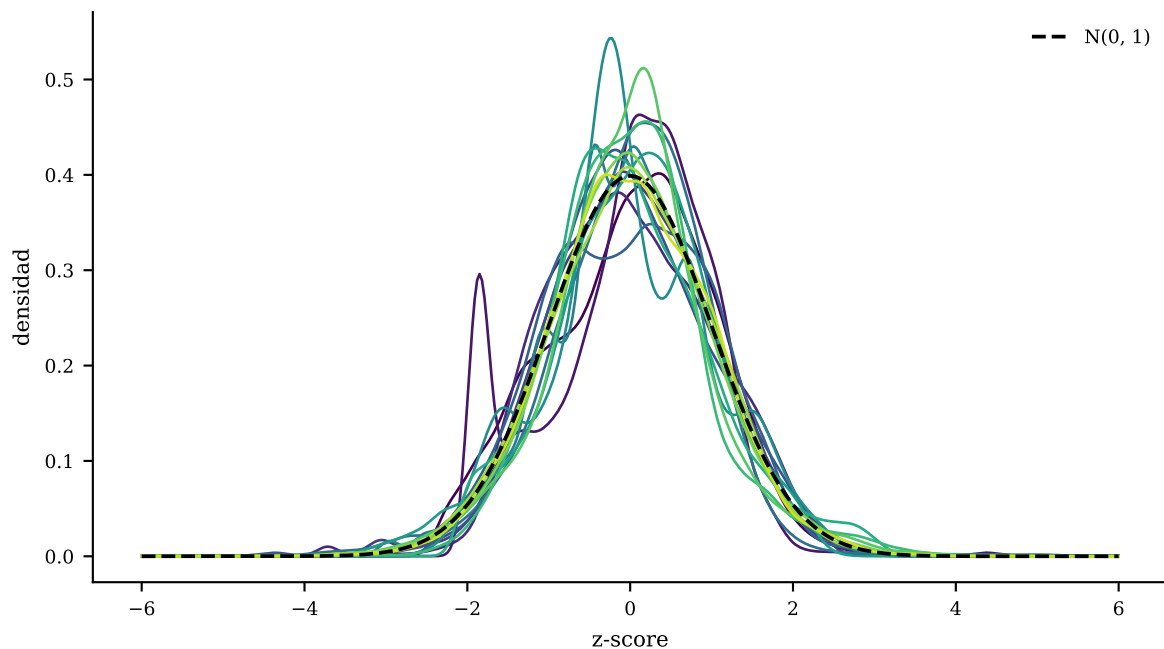


Figura 2: Densidad de las 16 continuas ya transformadas y estandarizadas.

Las 16 densidades quedan centradas y con dispersión comparable, que es lo que pedía el escalamiento. La normal de referencia muestra que varias siguen con el pico más alto de lo que correspondería —el residuo de curtosis que Box-Cox no corrige—, pero ninguna conserva la asimetría marcada del punto de partida.

```
fig, axs = plt.subplots(4, 4, figsize=(6.3, 7.4))
for a, col in zip(axs.ravel(), CONTINUAS):
    v = T[col].dropna().to_numpy()
    osm, osr = stats.probplot(v, dist="norm", fit=False)
    # rasterized: son 22 mil puntos por panel; en vectorial el PDF pesa 2 MB y
    # reports/figuras/ se versiona. La diagonal y los ejes siguen siendo vector.
    a.plot(osm, osr, ".", ms=.8, color=AZUL, alpha=.4, rasterized=True)
    lim = [min(osm.min(), osr.min()), max(osm.max(), osr.max())]
    a.plot(lim, lim, color=ROJO, lw=.8)
    a.set_title(col, fontsize=7.5)
    a.tick_params(labelsize=6)
fig.supxlabel("cuantiles teóricos", fontsize=8)
fig.supylabel("cuantiles observados", fontsize=8)
fig.tight_layout()
fig.savefig(FIGS / "transf_diaria_qqplots.pdf", dpi=200); plt.show()
```

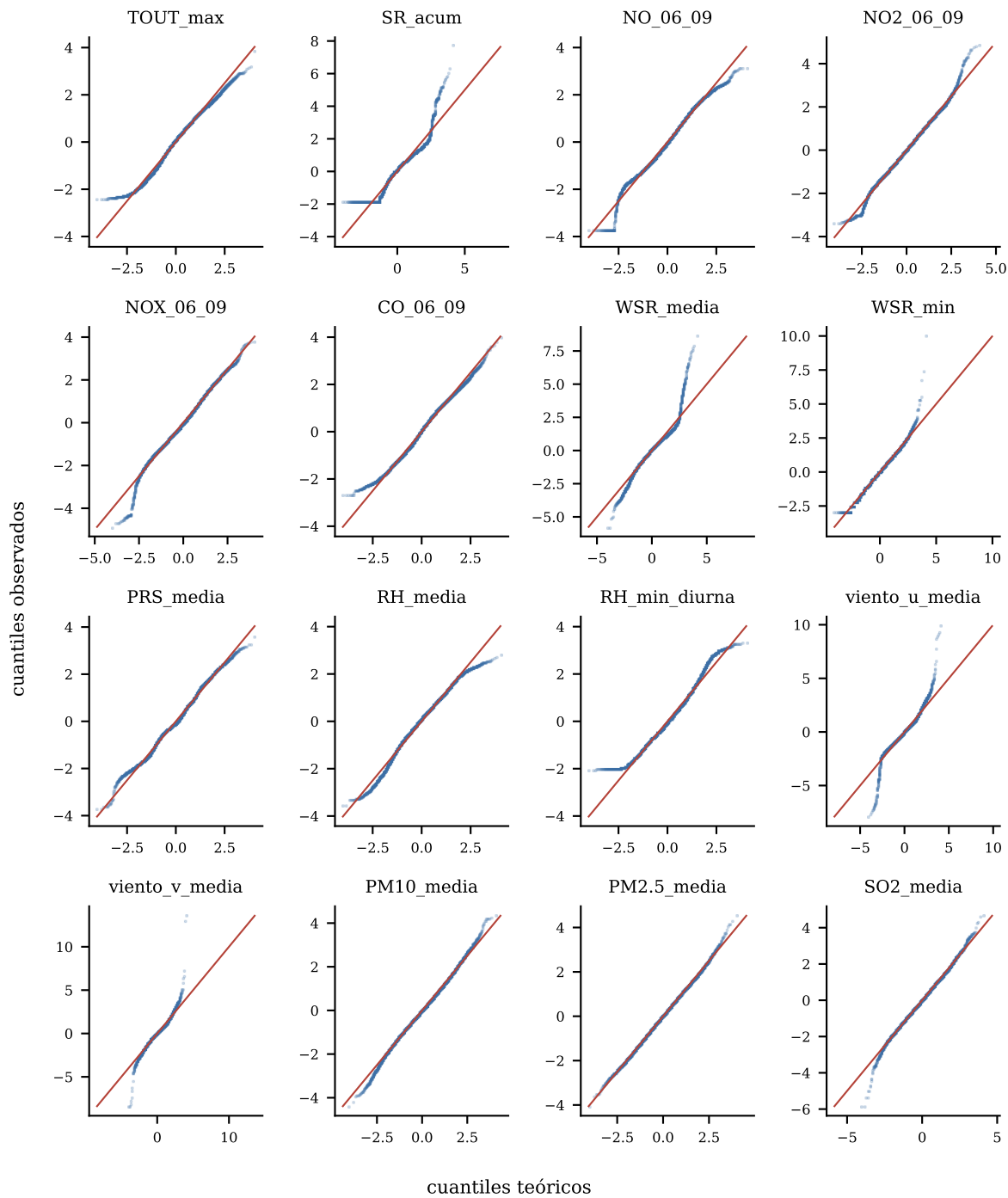


Figura 3: Q-Q normal de cada variable transformada y estandarizada.

Los Q-Q confirman lo que dice la tabla: el cuerpo central se alinea con la diagonal en las 16, y lo que se despega son los extremos de `WSR_media`, `viento_u_media` y `viento_v_media` —las tres del bloque de viento.

7. Salida

```
salida = PROC / "sima_transformado_diario.parquet"
T.to_parquet(salida, index=False)

ruta_params = PROC / "transformacion_diaria_parametros.csv"
params.round(6).to_csv(ruta_params)
print(f"{salida.name}: {T.shape[0]:,} filas × {T.shape[1]} columnas")
print(f"{ruta_params.name}: lambda y desplazamiento por variable, para invertir")
```

sima_transformado_diario.parquet: 26,691 filas × 34 columnas
transformacion_diaria_parametros.csv: lambda y desplazamiento por variable, para invertir

El parquet conserva las 34 columnas de #51 con las 16 continuas reemplazadas por su versión transformada. El CSV de parámetros guarda λ y desplazamiento: sin ellos la transformación no es invertible y los *loadings* del PCA no se pueden leer en unidades físicas.

8. Criterios de aceptación

OK	Contiene NO, SO ₂ , RH y SR (NO_06_09, SO ₂ _media, RH_media, SR_acum)
OK	Las 16 continuas cumplen $ \text{sesgo} < 0.5$
OK	Toda continua queda estandarizada (media 0, sd 1)
OK	Binarias y categóricas intactas
OK	Objetivo en escala física (MDA8, O ₃ _max_1h)
OK	No se perdieron ni se inventaron filas
OK	El patrón de faltantes no cambió

9. Qué queda para el diccionario

reports/secciones/diccionario_transformadas.tex describe hoy el conjunto **horario** (640,583 × 30). Debe reescribirse para el diario: 26,691 × 34, la tabla de sesgo y curtosis antes/después de §3, y la distinción entre las 16 continuas estandarizadas y las 18 columnas que se conservan en escala original. Los números están en transformacion_diaria_parametros.csv.

Queda pendiente de decisión del equipo qué hacer con sima_transformado_horario.parquet, que .gitignore sigue exceptuando: si el diario lo reemplaza como entrada del modelado, hay que quitarlo del repo, y son 21 MB que el equipo lleva semanas viendo ahí. No se toca sin avisarlo en la issue.

Uso de IA en este notebook

Se usó Claude Code como asistente en tres frentes: (1) portar a Python el método que #35 escribió en R —equivalencias de moments/MASS a scipy.stats.skew, kurtosis y boxcox—, conservando el criterio y el desplazamiento originales para que las dos notebooks sean comparables; (2) explorar el dataset diario antes de escribir la narrativa, que es de donde salieron los tres hallazgos que no

estaban en la issue: que no todas las continuas incumplen el criterio, que la predicción sobre el viento se cumple a medias, y que la cola más pesada de la tabla es `WSR_min` y no el viento vectorial; y (3) redactar las figuras y los chequeos de §8. Cada cifra citada en el texto se verificó corriendo el código contra `sima_diario_modelado.parquet` antes de escribirla, y las decisiones de alcance —no transformar `msnm`, no estandarizar MDA8, no tocar los extremos de `WSR` porque son alcance de #13— se discutieron y decidieron en conversación.